Sidecar Pattern: Bringing Theory to Practice with an Example

In this section, we'll bring the Sidecar Pattern to life, showing you just how it operates in a real-world scenario.

Creating the Main Application

First things first, we need an application. Let's create a simple Java web application. This application will expose a RESTful API and will serve as our main application.

Here is a simple Java web service using Spring Boot:

```java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class MainApplication {

  public static void main(String[] args) {
    SpringApplication.run(MainApplication.class, args);
  }

  @GetMapping("/api")
  public String getData() {
    return "Hello, Sidecar!";
  }
}
```

This web service will return the string "Hello, Sidecar!" whenever we hit the "/api" endpoint.

Introducing the Sidecar

Now that we have our main application, it's time to introduce the sidecar. In our case, let's create a sidecar that will log every request to our "/api" endpoint. For simplicity, our sidecar will also be a Java application:

```java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class LoggingSidecar {

  public static void main(String[] args) {
    SpringApplication.run(LoggingSidecar.class, args);
  }

  @GetMapping("/log")
  public String logRequest() {
    System.out.println("Request received at /api endpoint");
    return "Logged!";
  }
}
```

This sidecar will print a log message every time the "/log" endpoint is hit. We'll use this endpoint to log requests to our main application's "/api" endpoint.

Making the Sidecar and Main Application Work Together

Now, let's make our main application and sidecar work together. We need to modify our main application to call the "/log" endpoint of the sidecar whenever the "/api" endpoint is hit.

We'll use `RestTemplate`, a Spring utility class for calling RESTful services:

```java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@RestController
public class MainApplication {

  private RestTemplate restTemplate = new RestTemplate();

  public static void main(String[] args) {
    SpringApplication.run(MainApplication.class, args);
  }

  @GetMapping("/api")
  public String getData() {
    ResponseEntity<String> response =
restTemplate.getForEntity("http://localhost:8081/log", String.class);
    System.out.println("Response from sidecar: " + response.getBody());
    return "Hello, Sidecar!";
  }
}
```

Now, whenever the "/api" endpoint of our main application is hit, it will call the "/log" endpoint of our sidecar. The sidecar will log the request, and the main application will continue to process the request.

Keep in Mind: The Docker Factor

Remember, the sidecar and the main application need to be deployed within the same container to achieve the benefits of the Sidecar Pattern. While our Java example didn't cover this aspect, in a production scenario, you'd use a container platform like Docker to deploy the main application and the sidecar in the same container.

You'd also likely use Kubernetes as an orchestration tool to manage the deployment and scaling of your application and its sidecar. Kubernetes also has built-in support for sidecar containers, making it an ideal tool for implementing the Sidecar Pattern.

← Previous
The Architecture Of The Sidecar Pattern

Next →
Performance Implications

☐ Mark as Completed